MINI PROYECTO

TechSupport Solutions

Mini API de gestión de incidentes con ASP.NET Core Web API (.NET 10). Versión reducida de la guía integradora.

Tecnología	**Producto**
Web API · EF Core · SQLite · JWT	API funcionando y probada

1. El caso en breve

TechSupport Solutions da soporte técnico a pequeñas y medianas empresas. Hoy recibe las solicitudes por WhatsApp, llamadas y correos, y la gerencia detectó estos problemas:

- Las solicitudes se **pierden** entre cientos de mensajes.
- No hay asignaciones claras: dos técnicos atienden lo mismo y otro problema queda sin atender.
- Algunos empleados **consultan información que no les corresponde**.
- Cuando ocurre un error no se sabe qué pasó, y el sistema puede mostrar detalles internos.

META

Construir una **Web API** que centralice los incidentes, controle quién puede hacer qué y responda con errores seguros. Se prueba con Swagger, Postman o un archivo `.http` (lo que indique tu docente).

2. Reglas del juego

- **Justifica.** Cada decisión lleva una frase de “por qué” y “qué pasaría si no”. “Porque sí” vale cero.
- **Demuestra.** Si dices que funciona, muestra la petición y la respuesta HTTP.
- **Usa Git.** Mínimo 4 commits (2 por sesión). Un solo commit al final es una señal de alerta.
- **Entiende tu código.** Puedes consultar documentación o IA, pero te pueden pedir explicar cualquier línea.

3. Diseño base (ya decidido para ahorrar tiempo)

3.1 Roles y permisos

Operación	Administrador	Técnico	Usuario
Crear incidente	Sí	Sí	Sí
Ver incidentes	Todos	Solo los **asignados a él**	Solo los **que él creó**
Asignar técnico	Sí	No	No
Cambiar estado	Sí	Solo los asignados a él	No
Eliminar incidente	Sí	No	No

Con esto ya se resolvió la ambigüedad de “consultar propios”: para el técnico son los **asignados**, para el usuario son los **que creó**.

3.2 Ciclo de vida del incidente

```
Pendiente → Asignado → En proceso → Resuelto → Cerrado
```

- Al asignar un técnico, el incidente pasa de *Pendiente* a *Asignado*.
- El estado solo avanza **al siguiente**: no se puede cerrar un incidente sin resolverlo ni saltarse pasos.
- Solo se asigna a un usuario con rol **Técnico**.
- Un incidente *Cerrado* no se reabre (si el problema vuelve, se crea uno nuevo).

3.3 Modelo de datos mínimo

Tabla	Campos
Usuario	Id, NombreUsuario, PasswordHash, Rol
Incidente	Id, Titulo, Descripcion, Prioridad (Baja/Media/Alta/Crítica), Estado, UsuarioId, TecnicoId (nulo), FechaCreacion, FechaResolucion (nula)

Pista: `Incidente` se relaciona dos veces con `Usuario` (quien reporta y quien atiende). EF Core necesita que le indiques cuál propiedad corresponde a cada relación.

3.4 Endpoints a construir

Método	Endpoint	Roles	Qué hace
POST	/api/auth/login	Anónimo	Devuelve un JWT
POST	/api/incidentes	Todos	Crea incidente (estado Pendiente)
GET	/api/incidentes	Todos*	Lista según el rol
GET	/api/incidentes/{id}	Todos*	Detalle, solo si le corresponde
PUT	/api/incidentes/{id}/asignar	Administrador	Asigna técnico
PUT	/api/incidentes/{id}/estado	Admin, Técnico*	Avanza el estado
DELETE	/api/incidentes/{id}	Administrador	Elimina incidente

** Con la regla de propiedad de la sección 3.1.*

4. Sesión 1 (45 min): análisis y base del sistema

Minutos	Bloque	Qué haces
0–5	Arranque	Lees la guía, creas el repositorio Git y el proyecto.
5–15	Análisis (en papel)	Tabla problema → requerimiento → prueba, diagrama de estados y 2 decisiones.
15–40	Construcción	Modelos, `AppDbContext`, usuarios iniciales y endpoints de crear y listar.
40–45	Cierre	Pruebas rápidas y primer commit de funcionalidad.

Bloque A · Análisis (10 min)

A1. Del problema al requerimiento. Completa una tabla con los 4 problemas del caso:

Problema	Requerimiento (qué funcionalidad lo resuelve)	Prueba de que se resolvió
Las solicitudes se pierden		
Asignaciones poco claras		
Se consulta información ajena		
Errores sin registro / con detalles internos		

A2. Diagrama de estados. Dibuja los 5 estados y las transiciones permitidas. Marca la que el enunciado prohíbe explícitamente.

A3. Dos decisiones, una línea de justificación cada una.

- Cuando alguien consulta un incidente que no es suyo, ¿respondes **403** o **404**? ¿Qué ventaja de seguridad tiene el 404? Usa el mismo criterio en toda la API.
- ¿Puede un administrador eliminar un incidente que ya está *Cerrado*? ¿Qué se pierde si lo hace?

Bloque B · Construcción (30 min)

Crea la solución e instala los paquetes:

```
dotnet new webapi --use-controllers -n TechSupport.API
cd TechSupport.API
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package BCrypt.Net-Next
```

Luego, en este orden:

- **Modelos** `Usuario` e `Incidente` (con enums para Estado, Prioridad y Rol).

- **AppDbContext** con la configuración de las dos relaciones, y `Database.EnsureCreated()` al iniciar.
- **Usuarios iniciales** al arrancar, si la tabla está vacía: un administrador, un técnico y un usuario, con la contraseña guardada como **hash BCrypt**, nunca en texto plano.
- **DTOs** de entrada y salida. Nunca devuelvas la entidad `Usuario`: filtraría el `PasswordHash`.
- **POST y GET** de incidentes (aún sin reglas por rol; llegan en la sesión 2).

ATENCIÓN
Sin autenticación todavía, **todos** los endpoints están abiertos. Es normal en este punto, pero no lo olvides: es justo lo que arreglas en la sesión 2.

HECHO CUANDO
Se termina la sesión 1 cuando: puedes crear un incidente y listarlo desde Swagger/Postman, los datos quedan en el archivo SQLite y tienes al menos 2 commits.

5. Sesión 2 (45 min): seguridad, reglas y errores

Minutos	Bloque	Qué haces
0–15	JWT y roles	Login, token con claims, `[Authorize]` y restricciones por rol.
15–30	Propiedad e integridad	Casos críticos 1 y 3: quién puede ver/modificar y qué transiciones se permiten.
30–37	Errores	Caso crítico 2: middleware de excepciones.
37–45	Pruebas y cierre	Tabla de pruebas con evidencia, README y commit final.

Bloque A · JWT y roles (15 min)

- Configura en `appsettings` la clave, el emisor, la audiencia y la expiración del token (por ejemplo, 60 minutos).
- `POST /api/auth/login`: busca al usuario, verifica la contraseña con BCrypt y genera un token con tres claims: id (`NameIdentifier`), nombre (`Name`) y rol (`Role`).
- Si el usuario no existe o la contraseña es incorrecta, responde **401 con el mismo mensaje** en ambos casos. ¿Qué le revelaría a un atacante un mensaje distinto?
- En `Program.cs` registra la autenticación JWT y llama a `UseAuthentication()` **antes** de `UseAuthorization()`.
- Protege el controlador con `[Authorize]` y usa `[Authorize(Roles = "Administrador")]` en asignar y eliminar.

BUENA PRÁCTICA
En clase puedes dejar la clave del JWT en `appsettings.Development.json`. En un proyecto real **nunca** se sube a GitHub: se usa `dotnet user-secrets` en desarrollo y variables de entorno en producción. Anótalo en tu README.

Bloque B · Propiedad e integridad (15 min)

Tener un JWT válido **no basta**: autenticación responde "¿quién eres?" y autorización "¿qué puedes hacer con esto?". El rol por sí solo no resuelve el Caso 1, porque dos técnicos tienen el mismo rol pero distintos incidentes.

Caso crítico	Qué debes implementar	Evidencia mínima
1. Técnico modifica incidente ajeno (y usuario consulta información ajena)	El id de quien actúa se lee del **token**, nunca del cuerpo de la petición. Listar y detalle se filtran según la matriz 3.1. Cambiar estado solo si `TecnicoId` es el del técnico.	Petición de un técnico a un incidente de otro → 403 o 404 (según tu decisión A3).
3. Integridad de datos	No se asigna a quien no sea Técnico, el estado solo avanza al siguiente y `FechaResolucion` se llena al pasar a Resuelto.	Intentar cerrar un incidente Pendiente → 400 con mensaje claro.

> **CONCEPTO CLAVE**
> Escribe estas reglas **en un solo lugar reutilizable** (por ejemplo, un servicio `IncidenteService`), no copiadas en cada endpoint. Si cambian, las cambias una vez. Además, ocultar un botón nunca es seguridad: la API debe rechazar la petición aunque venga directa.

Bloque C · Errores seguros (7 min)

Caso crítico 2. Una excepción no controlada no debe mostrar rutas del servidor, nombres de base de datos ni detalles internos.

- Crea **un** middleware, `ExceptionHandlingMiddleware`, que capture la excepción, registre el detalle completo con `ILogger` y responda 500 con este JSON:

```
{
  "success": false,
  "message": "Ocurrió un error al procesar la solicitud.",
  "requestId": "a7d91c20",
  "timestamp": "2026-11-09T10:00:00"
}
```

- Usa `HttpContext.TraceIdentifier` como `requestId`: soporte lo busca en el log sin que el usuario vea nada sensible.
- Regístralo **al inicio** del pipeline. Explica en una línea qué excepciones no capturaría si estuviera al final.
- Para probarlo, crea un endpoint temporal que lance una excepción y **bórralo** al terminar.

6. Pruebas manuales con evidencia

Completa la columna de evidencia con una captura de la petición y la respuesta:

#	Prueba	Resultado esperado	Evidencia
1	Login con credenciales correctas	200 y token	
2	Login con contraseña incorrecta	401	
3	Consultar incidentes sin JWT	401	
4	Técnico intenta eliminar un incidente	403	
5	Crear incidente sin título	400	
6	Crear incidente correctamente	201	
7	Usuario consulta incidente de otro	403 o 404 (tu decisión A3)	
8	Cerrar un incidente que no está Resuelto	400	
9	La API produce una excepción	500 controlado, sin datos internos	

7. Entregables y criterios de aceptación

Entregables:

- Repositorio Git con al menos 4 commits.
- Proyecto `TechSupport.API` completo.
- Hoja de análisis (la foto de la hoja en papel es válida): tabla A1, diagrama A2 y decisiones A3.
- Tabla de pruebas con capturas.
- README corto: cómo ejecutar, usuarios de prueba y dónde va la clave del JWT en un proyecto real.

Marca solo lo que puedas demostrar:

- ☐ La API usa SQLite y Entity Framework Core.
- ☐ El login devuelve un JWT válido y las contraseñas están con hash.
- ☐ Los endpoints protegidos rechazan peticiones sin token.
- ☐ Los roles restringen asignar y eliminar.
- ☐ Técnicos y usuarios solo ven y modifican lo que les corresponde.
- ☐ No se cierra un incidente sin resolverlo ni se asigna a quien no es técnico.
- ☐ Los errores no revelan información interna y llevan un requestId.
- ☐ El proyecto se ejecuta siguiendo el README.

8. Evaluación

Criterio	Valor
Análisis breve (tabla, diagrama y decisiones)	15%
Modelo de datos, EF Core y endpoints	25%
JWT y roles	25%
Propiedad de recursos e integridad (casos 1 y 3)	20%
Manejo centralizado de errores (caso 2)	10%
Pruebas, Git y README	5%
Total	**100%**

> **ATENCIÓN**
> Si no puedes explicar tu propio código, la nota de los criterios afectados baja aunque el proyecto funcione. Entender es más importante que entregar.

9. Retos extra (si terminas antes)

- **Usuario desactivado con JWT vigente:** ¿por qué el token sigue siendo válido? Propón dos soluciones y compáralas.
- **Incidentes duplicados:** ¿qué pasa si el usuario pulsa Guardar tres veces? Investiga la idempotencia.
- **Logs en archivo:** configura Serilog con rotación diaria y evita subir la carpeta `Logs/` a GitHub.
- **Filtros y paginación:** `GET /api/incidentes?estado=Pendiente&page=1&pageSize=10`. ¿Qué haces si llega `pageSize=1000000`?
- **Pruebas automatizadas:** xUnit con `WebApplicationFactory` para el caso 1.
- **Segundo proyecto:** una interfaz MVC que consuma la API, sin acceso directo a la base de datos.